RAbHIT: R Antibody Haplotype Infrence Tool

Moriah Gidoni, Ayelet Peres, Gur Yaari

Last modified 2018-10-27

Contents

Introduction

Analysis of antibody repertoires by high throughput sequencing is of major importance in understanding adaptive immune responses. Our knowledge of variations in the genomic loci encoding antibody genes is incomplete, mostly due to technical difficulties in aligning short reads to these highly repetitive loci. The partial knowledge results in conflicting *V-D-J* gene assignments between different algorithms, and biased genotype and haplotype inference. Previous studies have shown that haplotypes can be inferred by taking advantage of *IGHJ6* heterozygosity, observed in approximately one third of the population([1],[2]).

Here we provide a robust novel method for determining *V-D-J* haplotypes by adapting a Bayesian framework, **RAbHIT**. Our method extends haplotype inference to *IGHD*- and *IGHV*-based analysis, thereby enabling inference of complex genetic events like deletions and copy number variations in the entire population. It calculates a Bayes factor, a number that indicates the certainty level of the inference, for each haplotyped gene.

More details can be found here:

Gidoni, Moriah, et al. "Mosaic deletion patterns of the human antibody heavy chain gene locus as revealed by Bayesian haplotyping." bioRxiv (2018): 314476.

Installation

To install `RAbHIT` please refer to https://bitbucket.org/yaarilab/rabhit

Input

RAbHIT requires two main inputs:

1. Pre-processed antibody repertoire sequencing data with heterozygosity in at least one gene
2. Database of germline gene sequences

Antibody repertoire sequencing data is in a data frame format. Each row represents a unique observation and columns represent data about that observation. The names of the required columns are provided below along with a short description.

Column Name	Description
`SUBJECT`	Subject name
`V_CALL`	(Comma separated) name(s) of the nearest V allele(s) (IMGT format)
`D_CALL`	(Comma separated) name(s) of the nearest D allele(s)
`J_CALL`	(Comma separated) name(s) of the nearest J allele(s)

An example dataset is provided with the `rabhit` package. It contains unique naive b-cell sequences, from a single individual.

The database of germline sequences should be provided in FASTA format with sequences gapped according to the IMGT numbering scheme ([3]). IGHV alleles in the IMGT database (build 201408-4) are provided with this package. (object name)

```
library(rabhit)
# Load example sequence data and example germline database
data(sample_db, HVGERM, HDGERM)
```

Pre-processing of the data

To get the most reliable result we suggest to follow the data pre-processing steps below.

1. Discover novel alleles (TIgGER [4])
2. Infer genotype and reassign sequences accordingly (TIgGER [4])
3. Sequence filtration by cells type:

- For naive cells sequences, use only V genes with <=3 mutation and no mutation in D gene.
- For PBMC cells sequences, first cluster sequences into clones (SHazaM [5]) then chose from each clone a representative sequence with the least number of mutations.

4. Preferably filter out non-functional sequences.

Running RAbHIT

The functions provided by this package can be used to perform any combination of the following:

1. Infer haplotype by anchor gene
2. Infer D/J single chromosome deletions by the V pooled approach
3. Infer double chromosome deletion by relative gene usage
4. Graphical output of the inferred haplotype
5. Graphical output of the inferred deletion

Infer haplotype by anchor gene

An individual's haplotype can be inferred using the functions `createFullHaplotype`. The function infers the haplotype based on the provided anchor gene. Using this function a contingency table is created for each gene, *from which strand is inferred for each allele*. The user can set the anchor gene for haplotyping as well as the column for which a haplotype should be inferred.

Prior to haplotyping, it is recommended to run TIgGER on the data, to detect new alleles and construct a genotype (TIgGER [4]).

```
# Inferred haplotype summary table
haplo_db <- createFullHaplotype(sample_db, toHap_col=c("V_CALL","D_CALL"),
                                hapBy_col="J_CALL", hapBy="IGHJ6",
                                toHap_GERM=c(HVGERM, HDGERM))
```

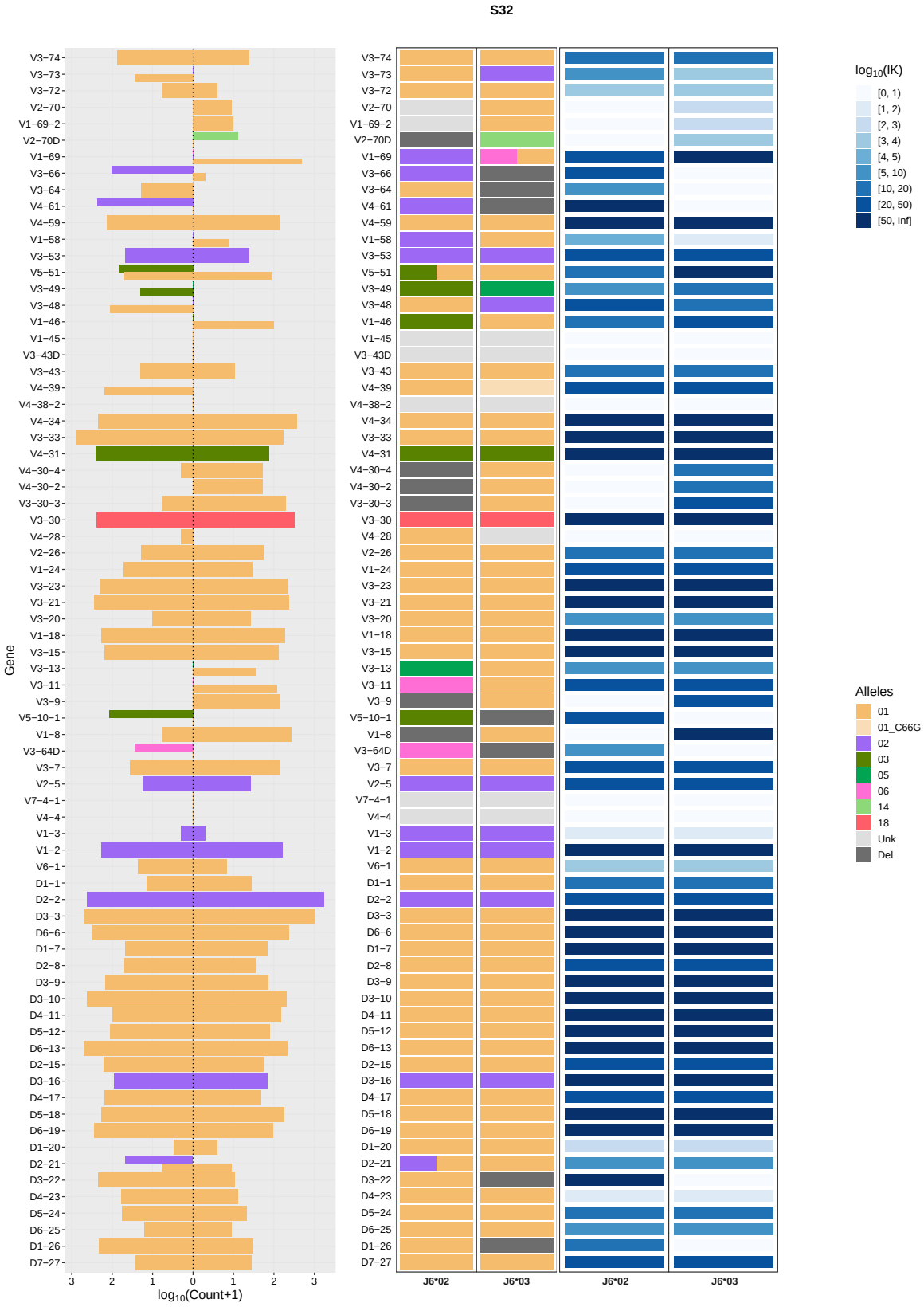
```
## In sample S32, 6475 sequnces were removed due to multiple assignments,
##  40108 sequences left.
```

```
head(haplo_db)
```

```
##   SUBJECT      GENE MinorFraction DoubleAllele IGHJ6_02 IGHJ6_03  ALLELES
## 1     S32   IGHV3-7             1            0       01       01       01
## 2     S32  IGHV3-21             1            0       01       01       01
## 3     S32  IGHV1-69         0.161            1       02    01,06 01,02,06
## 4     S32  IGHV3-30             1            0       18       18       18
## 5     S32 IGHV3-64D             1            0       06      Del       06
## 6     S32   IGHV1-8             1            0      Del       01       01
##   PRIORS_ROW     PRIORS_COL COUNTS1                 MP1               K1
## 1  0.48,0.52              1  36,140    -11.590764430191 21.4730927217493
## 2  0.48,0.52              1 281,237  -0.365273971780928 320.493783339993
## 3  0.48,0.52 0.52,0.19,0.30   0,484    0.61480351663325 136.343448274159
## 4  0.48,0.52              1 244,324 -0.0986982802780455 321.574048607258
## 5  0.48,0.52              1    27,0     1.3316304859018  8.4332169128303
## 6  0.48,0.52              1   5,273     1.3598271537726 68.4445656255678
##   ND1 COUNTS2              MP2              K2  ND2 COUNTS3
## 1   0    <NA>             <NA>            <NA> <NA>    <NA>
## 2   0    <NA>             <NA>            <NA> <NA>    <NA>
## 3   0   146,0 1.54261282977404 45.601839602712    0   0,279
## 4   0    <NA>             <NA>            <NA> <NA>    <NA>
## 5   0    <NA>             <NA>            <NA> <NA>    <NA>
## 6   0    <NA>             <NA>            <NA> <NA>    <NA>
##                MP3               K3  ND3 COUNTS4  MP4   K4  ND4
## 1             <NA>             <NA> <NA>    <NA> <NA> <NA> <NA>
## 2             <NA>             <NA> <NA>    <NA> <NA> <NA> <NA>
## 3 1.25337339512454 78.5946736952282    0    <NA> <NA> <NA> <NA>
## 4             <NA>             <NA> <NA>    <NA> <NA> <NA> <NA>
## 5             <NA>             <NA> <NA>    <NA> <NA> <NA> <NA>
## 6             <NA>             <NA> <NA>    <NA> <NA> <NA> <NA>
```

```
# Plot the haplotype
```

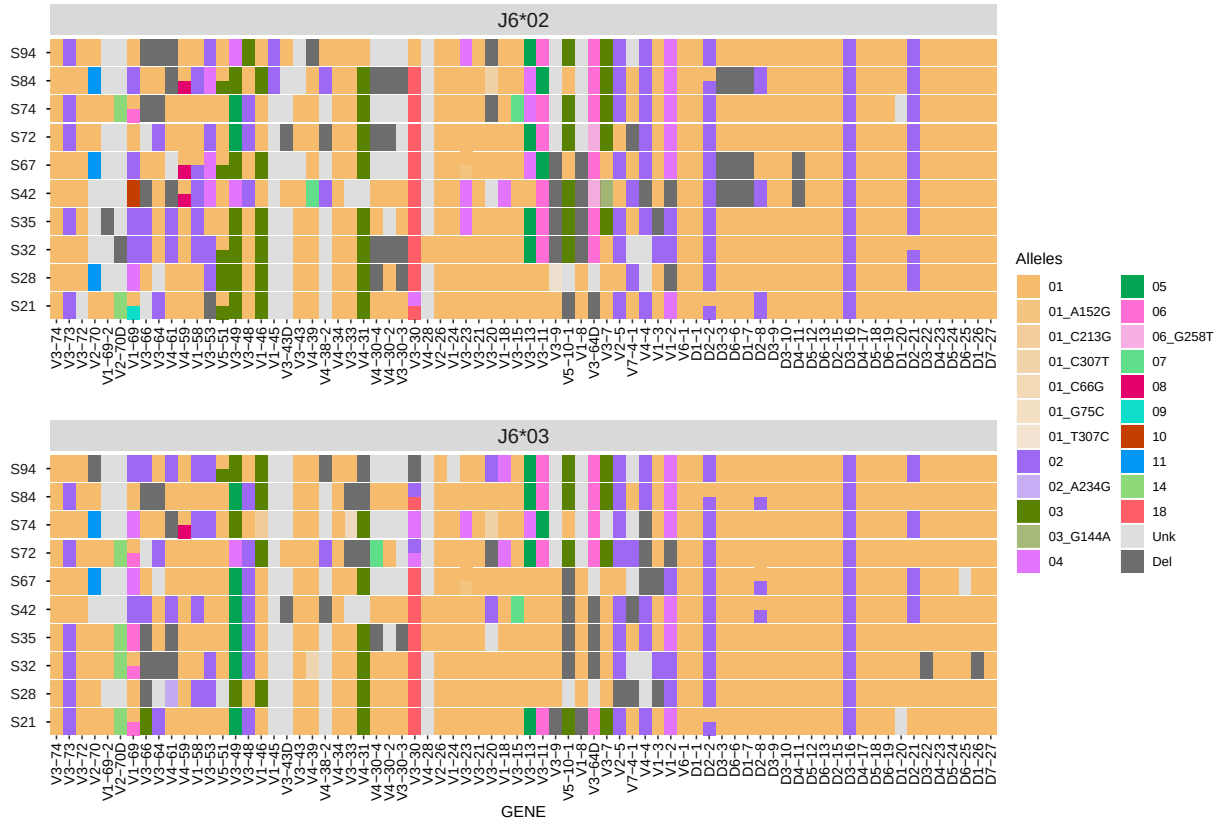
```
plotHaplotype(haplo_db)
```



```
# Plot interactive haplotype plot
p <- plotHaplotype(haplo_db, html_output = T)
# Save plot to html output
htmlwidgets::saveWidget(p, "haplotype.html", selfcontained = T)
```

For a group of individuals one can visualize the haplotype inferred using the `hapHeatmap` function. The function create a heatmap of the alleles genes inferred for each chromosome and.

```
# Load example sequence data
data(samples_db)
# Inferred haplotype summary table
haplo_db <- createFullHaplotype(samples_db, toHap_col=c("V_CALL","D_CALL"),
                                hapBy_col="J_CALL", hapBy="IGHJ6", toHap_GERM=c(HVGERM, HDGERM),
                                supress_print = T)
# Plot the haplotype inferred heatmap
hapHeatmap(haplo_db)
```



Infering double chromosome deletion by relative gene usage

Gene usage tends to change between individuals, in some cases the relative gene usage of certain individuals are much lower than the rest of the population. To asses whether the low frequency arise from a deleted gene, a binomial test described in Gidoni *et al.* (2018) ([6]) was implemented. They checked whether a certain relative gene usage of an individual is lower than the a chosen cutoff, for example for the *IGHV* genes, the chosen cutoff was 0.001. The `deletionsByBinom` function

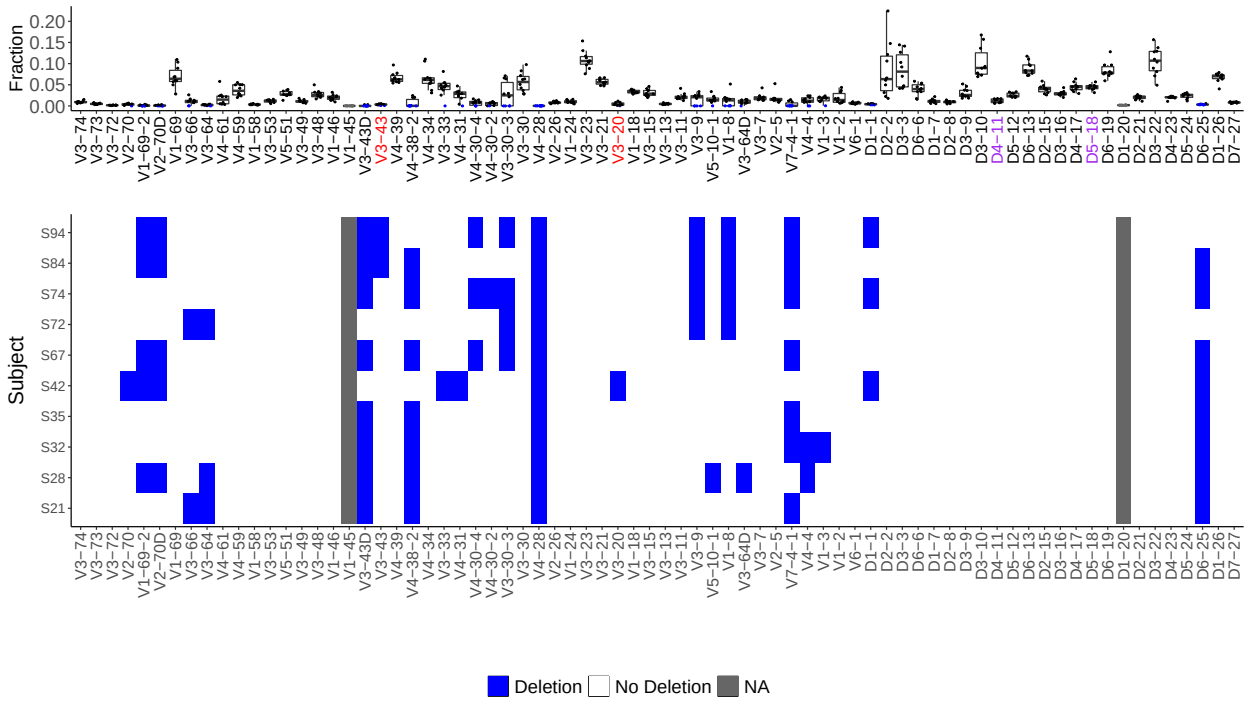
implements the binomial and return the detect gene deletion for a certain individual.

```
# Inferred deletion summary table
del_binom_db <- deletionsByBinom(samples_db)
head(del_binom_db)
```

```
## # A tibble: 6 x 6
##   SUBJECT GENE        FRAC  CUTOFF  PVAL DELETION
##   <chr>   <chr>      <dbl>   <dbl> <dbl> <fct>
## 1 S42     IGHV3-74 0.00714 0.00471     1 No Deletion
## 2 S94     IGHV3-74 0.0152  0.00471     1 No Deletion
## 3 S72     IGHV3-74 0.0105  0.00471     1 No Deletion
## 4 S21     IGHV3-74 0.00582 0.00471     1 No Deletion
## 5 S28     IGHV3-74 0.00793 0.00471     1 No Deletion
## 6 S84     IGHV3-74 0.00855 0.00471     1 No Deletion
```

For visualizing the deletion detected by `deletionsByBinom`, the `plotDeletionsByBinom` can be used. It is recommended to use this function for multiple individuals.

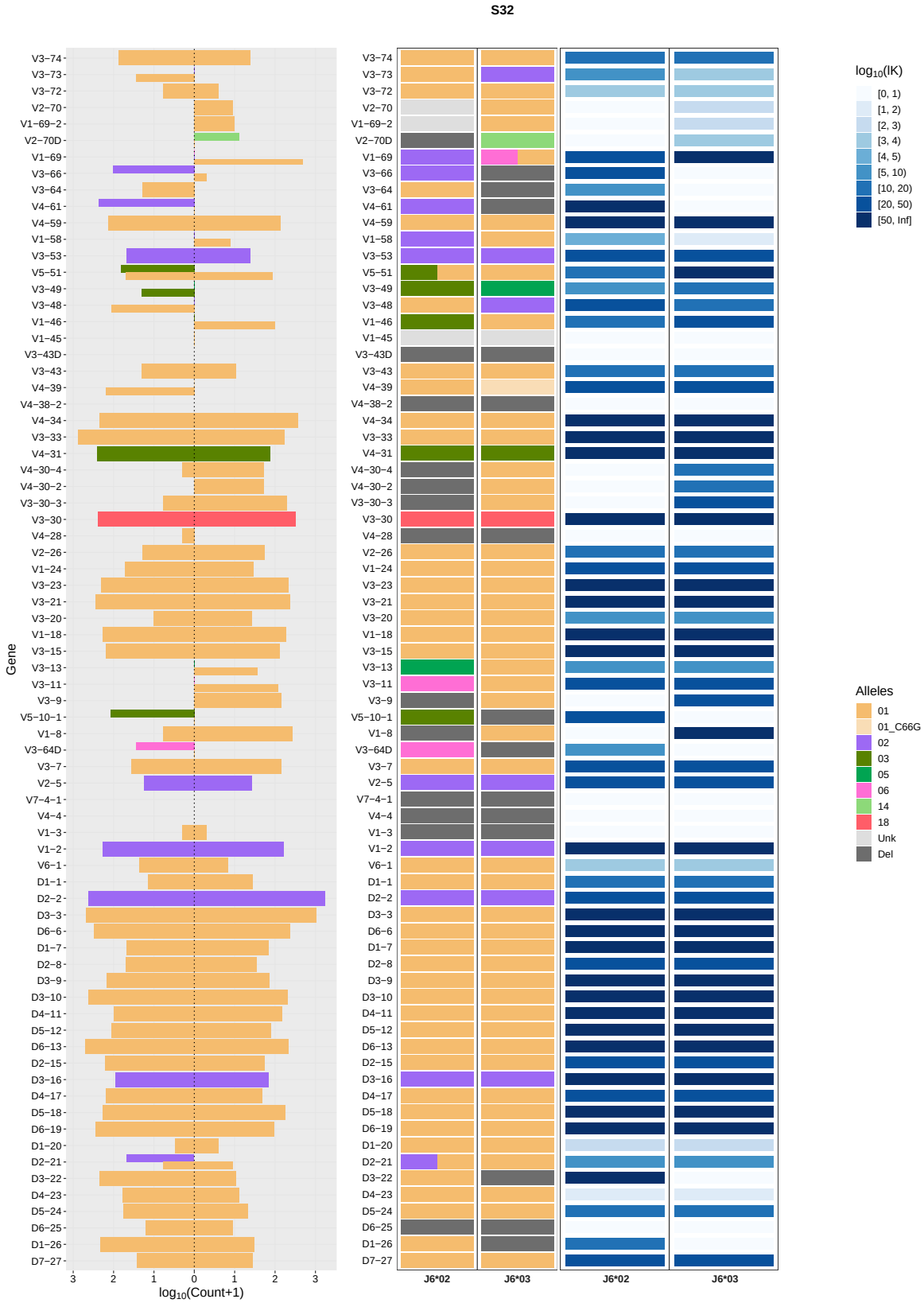
```
# Don't plot IGHJ
del_binom_db <- del_binom_db[grep('IGHJ',del_binom_db$GENE, invert = T),]
# Inferred deletion summary table
plotDeletionsByBinom(del_binom_db)
```



The detection of double chromosome deletion from the function can be then used for the haplotype inference, this prior knowledge of deletion can rise the certainty level of the inference of the genes where a deletion where a deletion was detected. The `createFullHaplotype` receives a vector of the deleted genes detected. V gene labels marked in red represent low expressed genes for which deletions are inferred with low certainty. D gene labels marked in purple represent indistinguishable

genes due to high sequence similarity, therefore alignment call is less reliable.

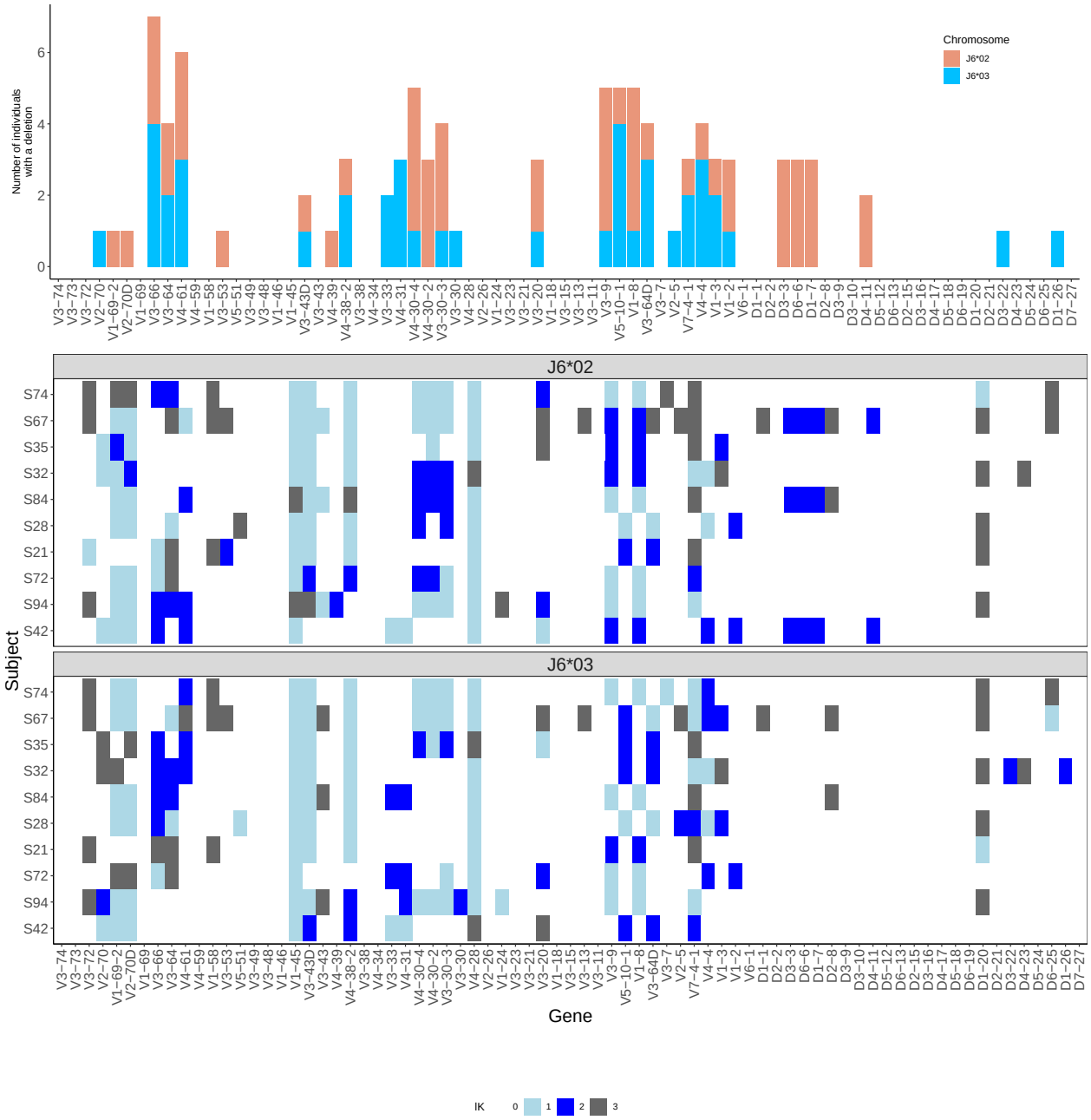
```
# Inferred deletion summary table
del_binom_db <- deletionsByBinom(sample_db)
haplo_db <- createFullHaplotype(sample_db, toHap_col=c("V_CALL","D_CALL"),
                        hapBy_col="J_CALL", hapBy="IGHJ6", toHap_GERM=c(HVGERM, HDGERM),
                        deleted_genes = del_binom_db, supress_print = T)
plotHaplotype(haplo_db)
```


S32
Gene
V3−74
V3−73
V3−72
V2−70
V1−69−2
V2−70D
V1−69
V3−66
V3−64
V4−61
V4−59
V1−58
V3−53
V5−51
V3−49
V3−48
V1−46
V1−45
V3−43D
V3−43
V4−39
V4−38−2
V4−34
V3−33
V4−31
V4−30−4
V4−30−2
V3−30−3
V3−30
V4−28
V2−26
V1−24
V3−23
V3−21
V3−20
V1−18
V3−15
V3−13
V3−11
V3−9
V5−10−1
V1−8
V3−64D
V3−7
V2−5
V7−4−1
V4−4
V1−3
V1−2
V6−1
D1−1
D2−2
D3−3
D6−6
D1−7
D2−8
D3−9
D3−10
D4−11
D5−12
D6−13
D2−15
D3−16
D4−17
D5−18
D6−19
D1−20
D2−21
D3−22
D4−23
D5−24
D6−25
D1−26
D7−27
3 2 1 0 1 2 3
log10(Count+1)
J6*02
J6*03
J6*02
J6*03
log10(IK)
[0, 1)
[1, 2)
[2, 3)
[3, 4)
[4, 5)
[5, 10)
[10, 20)
[20, 50)
[50, Inf]
Alleles
01
01_C66G
02
03
05
06
14
18
Unk
Del

Haplotype inference deletion heatmap

For a group of individuals, the deletions detected by the haplotyping process can be visualized with `deletionHeatmap`. The function create a heatmap of the deletion inferred and colors them by the certainty level (lK).

```
# Inferred haplotype summary table for multiple subjects
haplo_db <- createFullHaplotype(samples_db, toHap_col=c("V_CALL","D_CALL"),
hapBy_col="J_CALL", hapBy="IGHJ6", toHap_GERM=c(HVGERM, HDGERM), supress_print = T)
# plot deletion heatmap
deletionHeatmap(haplo_db)
```



Infering D/J single chromosome deletion by V pooled approach

Since V gene heterozygosity is extremely common, using V genes as anchors for haplotype inference could dramatically increase the number of people for which D haplotype can be inferred. However, reliable haplotype inference using V genes as anchors requires a much greater sequencing depth than haplotype inference using J6 gene as an anchor.

The RAbHIT package offers a solution to overcome the low number of sequences that connect a given V-D allele pair. The package function applies an aggregation approach, in which information

from several V heterozygous genes can be combined to infer D gene deletions.

The `deletionsByVpooled` function uses the V pooled approach to detect single chromosomal deletion for D and J.

```
# Inferred deletion summary table
del_db <- deletionsByVpooled(samples_db)
```

```
## [1] "5093 sequnces were removed due to multiple assignments, 37749 sequences left"
## [1] "The following genes used for pooled deletion detection"
##  [1] "IGHV3-23" "IGHV3-15" "IGHV1-18" "IGHV2-5"  "IGHV3-53" "IGHV4-39"
##  [7] "IGHV1-69" "IGHV3-7"  "IGHV3-11" "IGHV3-49"
## [1] "5774 sequnces were removed due to multiple assignments, 29869 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-23" "IGHV3-48" "IGHV5-51" "IGHV1-18" "IGHV3-49" "IGHV1-46"
## [7] "IGHV3-73"
## [1] "5374 sequnces were removed due to multiple assignments, 34009 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV1-18"  "IGHV1-46"  "IGHV3-64D" "IGHV3-49"  "IGHV2-5"
## [1] "5884 sequnces were removed due to multiple assignments, 29681 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-30" "IGHV3-48" "IGHV3-7"  "IGHV3-11" "IGHV3-49" "IGHV1-46"
## [1] "3954 sequnces were removed due to multiple assignments, 28840 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-9"  "IGHV1-69" "IGHV3-73"
## [1] "8991 sequnces were removed due to multiple assignments, 44838 sequences left"
## [1] "The following genes used for pooled deletion detection"
##  [1] "IGHV3-30"   "IGHV3-48"   "IGHV3-49"   "IGHV4-59"   "IGHV5-10-1"
##  [6] "IGHV5-51"   "IGHV3-53"   "IGHV3-11"   "IGHV1-69"   "IGHV2-70"
## [11] "IGHV3-73"   "IGHV1-58"
## [1] "5736 sequnces were removed due to multiple assignments, 40847 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-11" "IGHV4-39" "IGHV1-46" "IGHV5-51" "IGHV3-48" "IGHV3-13"
## [7] "IGHV3-49" "IGHV3-73"
## [1] "5072 sequnces were removed due to multiple assignments, 33204 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-23" "IGHV3-48" "IGHV1-46" "IGHV3-49" "IGHV3-7"  "IGHV3-11"
## [7] "IGHV1-58" "IGHV3-13"
## [1] "1212 sequnces were removed due to multiple assignments, 8737 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-53" "IGHV3-48" "IGHV3-11" "IGHV1-46" "IGHV1-69" "IGHV2-5"
## [7] "IGHV3-49"
## [1] "3624 sequnces were removed due to multiple assignments, 22412 sequences left"
## [1] "The following genes used for pooled deletion detection"
## [1] "IGHV3-33"   "IGHV3-30"   "IGHV3-15"   "IGHV1-46"   "IGHV3-48"
## [6] "IGHV3-73"   "IGHV3-49"   "IGHV2-70"   "IGHV5-10-1"
```

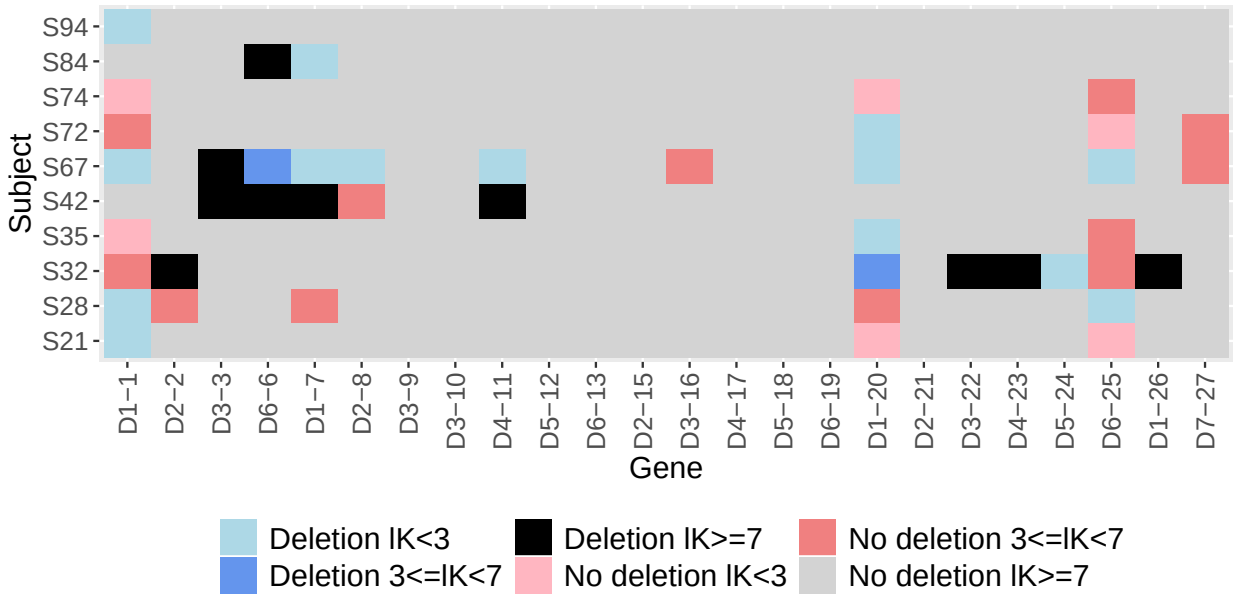
```
head(del_db)
```

```
##       GENE DELETION   K1_NEW COUNTS1_NEW    V_GENE     K1 SUBJECT
```

```
## 1  IGHD1-1        0 19.08503       15,22 V(pooled)  19.09     S42
## 2 IGHD1-20        0 12.03093       10,17 V(pooled)  12.03     S42
## 3 IGHD1-26        0 39.61305     330,564 V(pooled)  39.61     S42
## 4  IGHD1-7        1  8.22609        7,68 V(pooled)   8.23     S42
## 5 IGHD2-15        0 198.5908     170,309 V(pooled) 198.59     S42
## 6  IGHD2-2        0 279.3872     234,405 V(pooled) 279.39     S42
```

For visualizing the deletion detected by `deletionsByVpooled`, the `plotDeletionsByVpooled` can be used. However, it is recommended to use this function for multiple individuals.

```
# Plot the deletion heatmap
plotDeletionsByVpooled(del_db)
```



Contact

For help, questions, or suggestions, please contact:

- Moriah Gidoni
- Ayelet Peres
- Gur Yaari
- Issue tracker

References

1. Kidd *et al.* (2012).
2. Kirik *et al.* (2017).
3. Lefranc *et al.* (2003)
4. Gadala-Maria and Gidoni *et al.* (2018)
5. Gupta *et al.* (2015)

6. Gidoni *et al.* (2018).